

CMSSDiracAux

Bridging WMCore Workflows and DIRAC Execution Models

Todor Ivanov

March 16, 2026

1. Motivation

CMS workflows today are deeply coupled to the CMS submission infrastructure.

- ▶ WMCore
 - ▶ Defines the workflow object model: requests, tasks, steps, and policy metadata.
- ▶ WMAgent
 - ▶ Interprets WMCore workflows operationally and turns them into executable work.
- ▶ WMBS
 - ▶ Performs bookkeeping and fine-grained data-to-job mapping.
- ▶ GlideinWMS
 - ▶ Provisions pilots into the CMS global pool.
- ▶ HTCondor
 - ▶ Schedules and matches jobs to the provisioned resources.

These pieces evolved together. A CMS workflow therefore does not only describe *what* to process; it also implicitly assumes *how* the CMS infrastructure will partition, package, and execute that processing.

2. Problem Statement

DIRAC is a widely used distributed workload management framework.

However:

CMS workflows and DIRAC workflows follow **different architectural assumptions**.

Main question of this work:

Research question

Can CMS workflows be translated into a DIRAC-compatible execution model?

The challenge is architectural rather than syntactic. Both systems execute distributed jobs, but they encode workflow structure, data partitioning, and runtime reconstruction differently. A successful bridge must therefore preserve semantics, not just rename objects.

3. CMS Computing Architecture

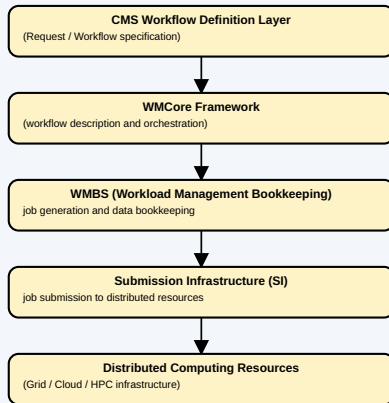
CMS workflows are defined through the **WMCore framework**.

Execution pipeline:

- ▶ RequestManager
- ▶ WMCore
- ▶ WMAgent
- ▶ WMBS
- ▶ Jobs

The request enters central services, WMCore builds the workflow representation, WMAgent operationalizes it, and WMBS turns policy into actual job partitions. The resulting jobs are then handed to the CMS submission stack.

Diagram



Source asset: [assets/cms-workflow-management-system_diagram01.pdf](#)

4. CMS Workflow Execution

CMS workflows are composed of:

- ▶ Tasks
 - ▶ Logical processing stages such as generation, reconstruction, or reduction.
- ▶ Steps
 - ▶ Concrete execution definitions, usually tied to a `cmsRun` context.
- ▶ Splitting definitions
 - ▶ Rules that partition input data into jobs using file, run, lumi, or event granularity.
- ▶ Runtime parameters
 - ▶ Job-specific settings such as masks, ranges, tweaks, and resource requirements.

A CMS workflow therefore already contains both semantic structure and operational policy. This is one reason interoperability is difficult: a target system must preserve both the high-level chain and the low-level job construction rules.

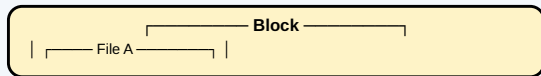
5. CMS Data Model

CMS data follows a hierarchical structure.

- ▶ Dataset
- ▶ Block
- ▶ File
- ▶ Run
- ▶ Lumisection
- ▶ Event

Storage entities and physics-content entities are not aligned perfectly. In particular, runs may span multiple files, while lumisections remain within files. This asymmetry is central to the splitting problem.

Diagram



Source asset: [assets/cms-data-model_diagram04.pdf](#)

6. CMS Splitting Model

CMS supports splitting at multiple levels:

- ▶ File splitting
 - ▶ Assigns whole files to jobs when file size and runtime are acceptable.
- ▶ Run splitting
 - ▶ Partitions work along run boundaries, even when runs cross file boundaries.
- ▶ Lumi splitting
 - ▶ Enables finer control and better runtime uniformity than file-based partitioning.
- ▶ Event splitting
 - ▶ The finest-grained mode, used when balancing or resource predictability requires it.

This fine granularity is exactly why CMS needs WMBS-like bookkeeping. The scheduler must know not only which files exist, but which parts of which files were assigned to which jobs. **Suggested diagram placement:** file/lumi/event partition examples from cms-splitting_diagram03-05.

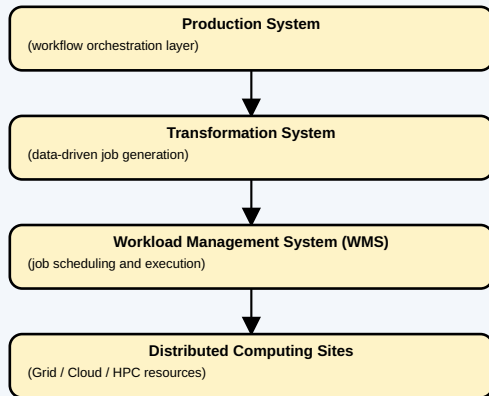
7. DIRAC Architecture

DIRAC is designed as a **general distributed workload management system**.

Core components include:

- ▶ Transformation system
 - ▶ Defines data-driven processing campaigns.
- ▶ Workload management system
 - ▶ Handles queues, matching, and execution control.
- ▶ Pilot system
 - ▶ Acquires resources first, then pulls payloads dynamically.
- ▶ Job agents
 - ▶ Run inside pilots and execute the assigned payload.

Diagram



Source asset: `assets/dirac-workflow-systems-architecture_diagram01.pdf`

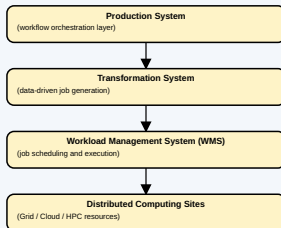
8. DIRAC Workflow Model

DIRAC workflows are expressed through **transformations**.

- ▶ Transformation
 - ▶ High-level processing definition linked to an input query or dataset view.
- ▶ Tasks
 - ▶ Units generated from the discovered input data and plugin logic.
- ▶ Jobs
 - ▶ Final execution payloads materialized from the tasks.

The important distinction is that DIRAC job generation is more naturally data-driven and runtime-oriented, whereas CMS precomputes much more of the workflow evolution before execution.

Diagram



9. DIRAC Pilot Model

DIRAC uses a pilot-based execution model.

- ▶ Pilot
 - ▶ A lightweight placeholder job that first acquires a worker slot.
- ▶ JobAgent
 - ▶ Requests a matching job once the pilot is running.
- ▶ Worker node
 - ▶ Executes the payload only after late binding has occurred.

This gives DIRAC flexibility in heterogeneous environments, but it also means the model expects dynamic assignment and centralized matching rather than pre-baked CMS-style job packages. **Suggested diagram placement:** pilot/JobAgent path from the hybrid integration report or a distilled version from DIRAC WMS notes.

10. CMS vs DIRAC Execution

The two systems differ significantly in workflow semantics.

Key differences include:

- ▶ splitting model
- ▶ workflow structure
- ▶ runtime packaging

CMS evolves a workflow explicitly through tasks and splitting rules before submission. DIRAC, by contrast, keeps more of the assignment and materialization dynamic and transformation-driven.

Diagram



Source asset: assets/cms-vs-dirac-execution-model_diagram01.pdf

11. Where the Mismatch Appears

Major mismatches occur in:

- ① Data splitting semantics
 - ▶ CMS can reason below file level; vanilla DIRAC fundamentally reasons at file level.
- ② Runtime environment packaging
 - ▶ CMS reconstructs part of the job context dynamically using workflow artifacts and CMSSW state.
- ③ Workflow structural assumptions
 - ▶ CMS task/step semantics do not map one-to-one to transformations and jobs.

These are exactly the places where direct translation breaks down. Every viable solution must explicitly compensate for them.

12. Key Architectural Insight

A central observation emerged during this work:

Critical point

WMBS exists largely because CMS requires splitting at run, lumi, and event granularity.

Therefore:

- ▶ Any alternative scheduling system must reimplement similar bookkeeping mechanisms.
- ▶ The missing functionality does not disappear when WMBS is removed; it reappears elsewhere.

This is arguably the most important conceptual result of the entire project. The interoperability problem is not about finding new names for old objects; it is about preserving the bookkeeping semantics that make CMS workflows executable at all.

13. Why Translation Is Difficult

CMS workflows encode assumptions about:

- ▶ job granularity
- ▶ runtime environment reconstruction
- ▶ dataset partitioning

DIRAC workflows assume:

- ▶ file-based job inputs
- ▶ generic runtime execution
- ▶ transformation-driven job creation

The models are **not isomorphic**. Because the assumptions differ structurally, a field-to-field mapping is insufficient. We need a mediation layer that preserves semantics independently of either implementation.

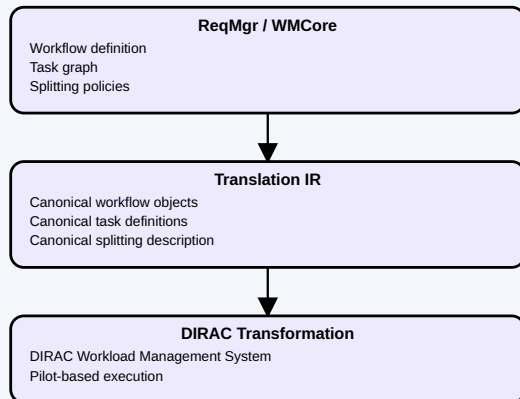
14. Translation IR Concept

To bridge the models we introduce an **Intermediate Representation (IR)**.

- ▶ WMCORE workflow semantics are extracted first.
- ▶ The IR stores canonical workflow, task, data, and splitting information.
- ▶ DIRAC-side materialization is performed from that canonical form.

The IR is not a convenience layer; it is an architectural necessity. It decouples semantic content from the infrastructure-specific way that WMCORE and DIRAC choose to represent and execute it.

Diagram



Source asset: [assets/translation-ir-design_diagram02.pdf](#)

15. Canonical Translation Objects

The IR represents workflows using canonical objects derived from WMCore.

- ▶ WMWorkload
 - ▶ Captures workflow-global identity and policy.
- ▶ WMTask
 - ▶ Becomes the main unit for transformation-level reasoning.
- ▶ WMStep
 - ▶ Encodes the executable step and runtime contract.
- ▶ WMSplitting
 - ▶ Supplies the rules that ultimately determine job boundaries.

Serializing these objects is what makes the PoC possible. Once they are externalized from opaque Python internals, they can be normalized, compared, and re-emitted into other execution models.

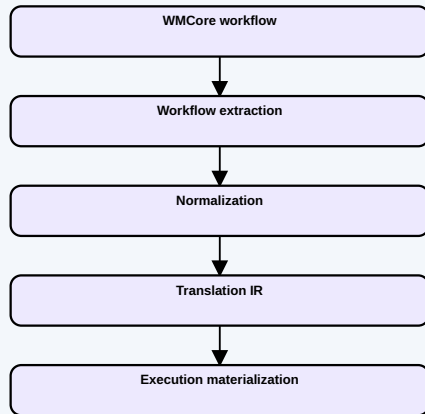
16. Translation Pipeline

The translation process follows several stages.

- ▶ Workflow extraction
- ▶ Normalization
- ▶ Translation IR
- ▶ DIRAC transformation

Each stage removes one layer of WMCore-specific coupling. Extraction captures the original workflow, normalization resolves shape and naming issues, the IR freezes the semantics, and materialization projects those semantics onto DIRAC concepts.

Diagram



Source asset: assets/translator-design_diagram01.pdf

17. Proof-of-Concept Pipeline

The CMSDiracAux prototype implements this pipeline.

- ▶ `wmcGet.py`
 - ▶ Fetches the workflow and serializes the WMCore objects.
- ▶ `wmc2transf.py`
 - ▶ Builds canonical translation objects and emits DIRAC-like artifacts.
- ▶ `runLocalTransformation.py`
 - ▶ Applies CMS splitting logic locally and generates tasks and task jobs.

This is enough to demonstrate end-to-end translation without requiring full server-side DIRAC deployment. The PoC is therefore controlled, inspectable, and reproducible.

18. Local Transformation Simulation

The PoC simulates the DIRAC transformation engine locally.

- ▶ transformations
 - ▶ Local JSON descriptions of DIRAC-style processing units.
- ▶ tasks
 - ▶ The result of applying the CMS splitting policy to the resolved inputs.
- ▶ job descriptions
 - ▶ Per-task job payloads ready for inspection and later runtime execution.

This local materialization stage is where the bridge becomes concrete: we can see exactly how a CMS workflow turns into a family of DIRAC-like jobs.

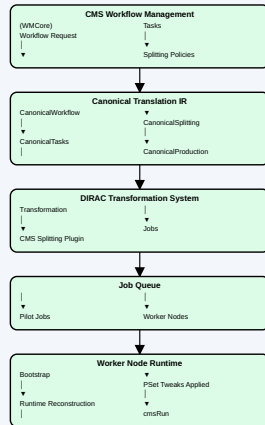
19. Job Materialization

Job creation follows the splitting rules defined in the workflow.

- ▶ Transformation
 - ▶ Holds the normalized task and data references.
- ▶ CMS splitting plugin
 - ▶ Reintroduces the WMBS-style job-to-data mapping logic.
- ▶ Jobs
 - ▶ Receive runtime parameters such as dataset, run range, lumi mask, and tweaks.

This is the place where DIRAC alone is not enough. The plugin is where CMS-specific semantics are reintroduced on the DIRAC side.

Diagram



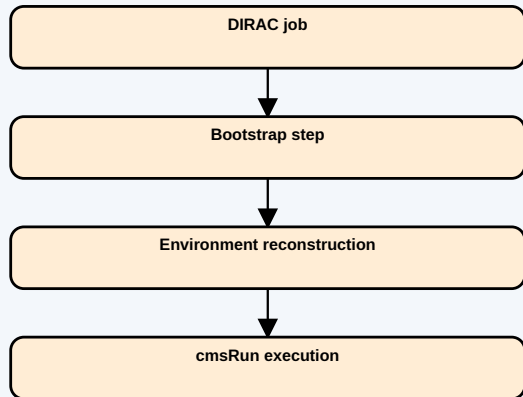
Source asset: assets/cmsdiracaux-main-architecture_diagram01.pdf

20. Runtime Reconstruction

Each job reconstructs the CMS runtime environment.

- ▶ **cmsRun**
 - ▶ Final execution entry point for CMS payloads.
- ▶ **CMSSW**
 - ▶ Provides the experiment software environment.
- ▶ **configuration files**
 - ▶ Drive the step-specific processing logic and module chain.
- ▶ **workflow parameters**
 - ▶ Bind the abstract workflow to one concrete dataset partition.

Diagram



Source asset: [assets/job-runtime-differences_diagram08.pdf](#)

21. Architectural Challenges

Key challenges identified during the study:

- ▶ sub-file splitting
 - ▶ CMS may require run/lumi/event-aware partitions even when DIRAC naturally reasons at file granularity.
- ▶ runtime environment packaging
 - ▶ CMS runtime artifacts are distributed at workflow and job granularity, not only through shared software areas.
- ▶ workflow structural differences
 - ▶ TaskChain and StepChain semantics are not native DIRAC objects.

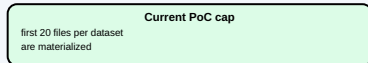
The PoC works by controlling the scope of these challenges rather than pretending they do not exist. That is why its limitations are informative rather than accidental.

22. Origins of the Current PoC Limitations

The current limitations are architectural in origin, not just implementation backlog.

- ▶ 20 files per dataset cap
 - ▶ Introduced because real CMS datasets can contain thousands of files, making local materialization too large for an early-stage PoC.
- ▶ local transformation simulation
 - ▶ Used because server-side DIRAC deployment, plugin registration, and production permissions were deliberately postponed.
- ▶ no full run/lumi realization end-to-end
 - ▶ Requires a mature CMS splitting plugin and equivalent bookkeeping beyond vanilla file-granularity DIRAC behavior.
- ▶ no production-grade sandbox/runtime solution
 - ▶ DIRAC and CMS make different assumptions about CVMFS, sandboxes, and job-level runtime artifacts.

Diagram



Source asset: `assets/architecture-diagram_diagram04.pdf`

Diagram placeholder

Suggested supporting
diagram: local-only path
through extraction →
materialization → task
jobs, without server-side
DIRAC deployment.

23. Execution Infrastructure

CMS execution infrastructure includes:

- ▶ GlideinWMS
 - ▶ Pilot provisioning into the CMS Global Pool.
- ▶ HTCondor
 - ▶ Scheduling, matching, and schedd-based execution control.
- ▶ site scheduling
 - ▶ CMS policy and classad-driven resource targeting across distributed sites.

DIRAC introduces a different pilot-driven model.

Integrating both requires careful architectural design. The objective is not to stack two workload managers on top of each other. The objective is to reuse GlideinWMS where CMS already has strong infrastructure, while letting DIRAC contribute transformation-driven workflow handling.

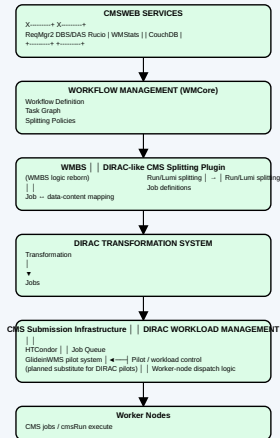
24. CMSDiracAux System Slice

The CMSDiracAux prototype connects the two ecosystems.

- ▶ WMCORE
- ▶ Translation IR
- ▶ DIRAC
- ▶ GlideinWMS
- ▶ HTCondor

The key cut is between CMS workflow definition and execution backend. CMSDiracAux intercepts the workflow before WMBS-driven job submission and re-materializes it through DIRAC-side abstractions.

Diagram



Source asset: assets/interoperability-architecture_diagram02.pdf

25. Hybrid Execution Model

The resulting architecture enables hybrid workflow execution.

- ▶ translated
 - ▶ WMCore workflows are first normalized into canonical semantics.
- ▶ materialized
 - ▶ DIRAC-compatible transformations and jobs are generated from that semantic layer.
- ▶ executed using DIRAC infrastructure
 - ▶ While still preserving the role of CMS submission infrastructure where appropriate.

while preserving CMS semantics. This is the heart of the proposal: not migration by deletion, but interoperability by controlled decomposition and recomposition.

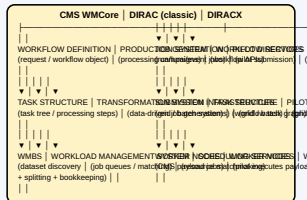
26. DIRACX Future

DIRAC is evolving toward **DIRACX**, a modernized architecture.

- ▶ service-based architecture
 - ▶ Better suited for external adapters and lighter integration services.
- ▶ token-based authentication
 - ▶ Aligns with current operational models and modern service security.
- ▶ improved extensibility
 - ▶ Makes it more realistic to externalize the queue/consumer pieces needed for hybrid CMS integration.

The DIRAC-to-DIRACX transition is strategically important. Even if the PoC is implemented with classic DIRAC assumptions today, the long-term path likely benefits from DIRACX-native integration components.

Diagram



27. CWL and Workflow Portability

Future integration may leverage CWL.

- ▶ WMCore
 - ▶ Supplies the original workflow semantics.
- ▶ IR
 - ▶ Acts as the experiment-aware semantic bridge.
- ▶ CWL
 - ▶ Offers a portable workflow description target.
- ▶ DIRACX
 - ▶ Becomes a candidate execution environment for the portable form.

The practical takeaway is that the IR has two downstream roles: immediate DIRAC materialization for the PoC, and future export into a more portable workflow language such as CWL.

28. Results

The proof-of-concept demonstrates:

- ▶ CMS workflows can be translated
 - ▶ WMCore objects can be serialized and normalized into a stable intermediate representation.
- ▶ DIRAC transformations can represent CMS jobs
 - ▶ Provided CMS-specific splitting semantics are reintroduced through plugins and runtime parameters.
- ▶ hybrid execution models are feasible
 - ▶ The architectural path toward DIRAC + CMS Global Pool integration is coherent and technically plausible.

The PoC is deliberately limited, but the limitations are exactly what make the conclusions trustworthy: they reveal where the real architectural boundaries are.

29. Architectural Conclusion

The key architectural result:

- ▶ Direct translation is not sufficient.
 - ▶ Native WMCore and DIRAC workflow models are structurally non-isomorphic.
- ▶ A semantic mediation layer is required.
 - ▶ The system must preserve workflow semantics independently of the original infrastructure.
- ▶ The Translation IR provides that layer.
 - ▶ It decouples CMS workflow meaning from DIRAC execution representation.

This is the central scientific claim of the project. The IR is not an implementation detail; it is the architectural device that makes interoperability possible.

30. Final Message

CMS workflows and DIRAC workflows follow different models.

However:

Take-away

Through a translation architecture and hybrid execution design, interoperability is achievable.

- ▶ Preserve CMS semantics explicitly.
- ▶ Materialize execution through DIRAC-compatible abstractions.
- ▶ Reuse CMS resource infrastructure where it already solves the right problem.

CMSDiracAux should be understood as an interoperability experiment, but also as a conceptual decomposition of workflow semantics from infrastructure. That is its broader contribution.

Appendix: Master Architecture Slide

Diagram

